Modbus Scanner + ECY Point Populator

Version 2.0.0

User Guide

Modbus TCP/RTU Register Scanner | ECY API Point Population Tool

Supports Modbus TCP (MBAP) and RTU-over-TCP Tunnel protocols

1. Overview

Modbus Scanner + ECY Point Populator 2.0.0 is a Windows desktop diagnostic and commissioning tool built for building automation and industrial control engineers. It provides a single interface to scan, read, write, and document Modbus devices over TCP or RTU-over-TCP tunnel connections, and to publish discovered register maps directly to an ECY REST API.

1.1 Key Capabilities

- Connect to any Modbus device via TCP/IP (MBAP) or RTU-over-TCP gateway (Moxa, Advantech, Digi, etc.)
- Scan all four Modbus register types in one click: Coils (FC01), Discrete Inputs (FC02), Holding Registers (FC03), Input Registers (FC04)
- Live polling with configurable interval for real-time monitoring
- Read and write individual coils and registers (FC05, FC06)
- Interpret register values as multiple data types: UINT16, INT16, UINT32, INT32, FLOAT32, HEX, Binary, Bit, Bool
- Frame Injector: craft and send custom Modbus frames in structured or raw hex mode
- Export and import register maps as CSV for documentation or re-use
- POST discovered devices and point lists directly to the ECY API
- Full debug log with raw byte traces, MBAP header decode, and RTU CRC validation

1.2 System Requirements

Feature / Field	Description
Operating System	Windows 10 or later (64-bit)
Runtime	PowerShell 5.1 or PowerShell 7+ (.NET WPF)
Network	TCP/IP access to the target Modbus device or gateway
Modbus Protocol	Modbus TCP (port 502 default) or RTU-over-TCP via serial gateway
ECY API (optional)	HTTPS access to ECY REST API endpoint for point population

2. Getting Started

Launch the application by double-clicking Modbus Scanner+ECY point populator 2.0.0.exe. The main window opens maximised with the left control panel and the register map grid on the right.

NOTE: On first launch, Windows may display a security prompt. Click Run anyway to proceed. The tool does not require administrator privileges for standard Modbus TCP operations.

2.1 Main Window Layout

Feature / Field	Description
Title Bar	Displays the application name and version. The Debug Mode checkbox in the top-right enables or disables verbose protocol logging.
Left Panel	Scrollable control panel containing all configuration sections: Connection, Scan Parameters, Known Register Counts, Write Register, Frame Injector, POST Device to API, and Tools.
Register Map Grid	Right-hand panel displaying discovered registers with columns for Address, Register Type, Data Type, Value, Name/Label, and Changed indicator.
Debug Log	Collapsible panel at the bottom of the register map area. Click the DEBUG LOG bar to expand or collapse. Displays raw bytes, MBAP headers, RTU frames, and diagnostic output.
Status Bar	Bottom strip showing connection state (green = connected, red = disconnected), connection details, and cumulative poll/error counters.

2.2 Connecting to a Device

Modbus TCP (Standard MBAP)

Use this mode for devices that natively speak Modbus TCP — PLCs, energy meters, and most modern Modbus-capable devices with an Ethernet port.

1. Ensure the Modbus TCP toggle at the top of the Connection section is active (highlighted blue).
2. Enter the device IP address or hostname in the Host / IP Address field.
3. Set the TCP Port (default: 502).
4. Set the Unit ID (Slave ID). This is commonly 1 but varies by device — consult the device manual.
5. Enter a Timeout in milliseconds (default: 2000 ms). Increase for slow or remote devices.
6. Optionally enter a Device Name — used when posting to the ECY API.
7. Click Connect. The status bar turns green on success.

TIP: If the connection fails, use the Network Diagnostic button to ping the device and test the port before troubleshooting further.

Modbus RTU over TCP Tunnel

Use this mode when your device communicates via serial RS-485 and is connected through a serial-to-Ethernet gateway such as a Moxa NPort, Advantech EKI, or Digi AnywhereUSB. The tool sends raw RTU frames (no MBAP header) over TCP, exactly as the gateway expects.

8. Click the Modbus RTU toggle button.
9. Click TCP Tunnel (IP) in the RTU sub-mode selector.
10. Enter the Gateway IP Address and TCP Port (often 4001 or 502 depending on gateway configuration).
11. Set the Unit ID to match the physical device address on the RS-485 bus.
12. Baud Rate, Parity, Data Bits, and Stop Bits are displayed for reference — these must be configured on the gateway itself, not within this tool.
13. Click Connect.

WARN: RTU Tunnel mode sends raw RTU frames including CRC over TCP with no MBAP wrapper. Ensure your gateway is configured in TCP transparent (passthrough) mode, not in Modbus TCP gateway mode, otherwise frames will be double-wrapped.

3. Scanning Registers

The Scan Parameters section controls which registers are read and how frequently. All scanning requires an active connection.

3.1 Modbus Register Types

FC	Register Type	Access	Address Range	Notes
FC01	Coils	Read / Write	00001 – 09999	1-bit digital outputs
FC02	Discrete Inputs	Read only	10001 – 19999	1-bit digital inputs
FC03	Holding Registers	Read / Write	40001 – 49999	16-bit analogue / config
FC04	Input Registers	Read only	30001 – 39999	16-bit analogue inputs

3.2 Running a Scan

14. In the Register Type dropdown, choose a specific FC or select ALL to scan all four types in one operation.
15. Set the Start Address (zero-based). The Modbus reference range preview updates automatically below the field.
16. Open the Known Register Counts section and set the quantity for each FC type. The tool reads exactly this many registers starting from the start address.
17. Set Poll Interval to 0 for a single one-shot scan, or enter a value in milliseconds (minimum 100) to enable continuous polling.
18. Click Scan. Results appear in the Register Map grid immediately.

NOTE: Coil and Discrete Input rows are always displayed regardless of value. Holding and Input Register rows with a value of zero are hidden by default to reduce noise — named rows are always retained.

3.3 Live Polling

When Poll Interval is set to a non-zero value, the tool automatically repeats the scan at the specified interval. The Changed column shows *** next to any register whose value has changed since the previous poll. Click Stop Poll to halt polling at any time.

3.4 Register Map Grid

The grid displays all discovered registers. Two columns are directly editable:

Feature / Field	Description

ADDRESS (Modbus)	The Modbus reference address (e.g. 40001 for holding register 0). Read only.
REGISTER TYPE	FC01 Coil, FC02 Discrete Input, FC03 Holding Register, or FC04 Input Register. Read only.
DATA TYPE	Click the cell to open a dropdown. Change the interpretation of the raw value. See Section 3.5 for all options.
VALUE	Decoded value based on the selected data type. Updates on each poll. Read only.
NAME / LABEL	Click any cell to type a custom label. Labels persist across re-scans and are included in CSV export.
CHANGED	Shows *** when the value has changed since the last scan. Cleared on the next poll.

Context Menu

Right-click any row in the grid to access:

- Remove Row — deletes the row from the map (does not affect the device).
- Encode / Decode Bits — opens the bit decode window for the selected register (see Section 3.6).
- Copy Value — copies the decoded value string to the clipboard.
- Copy Modbus Address — copies the Modbus reference address to the clipboard.

3.5 Data Type Interpretations

Feature / Field	Description
BIT	Displays ON (1) or OFF (0). Used for coil and discrete input registers.
BOOL	TRUE if non-zero, FALSE if zero.
UINT16	Unsigned 16-bit integer. Range: 0 to 65,535. Shown in decimal and hex.
INT16	Signed 16-bit integer. Range: -32,768 to 32,767. Shown in decimal and hex.
UINT32	Unsigned 32-bit integer formed from two consecutive registers (Hi word first). Range: 0 to 4,294,967,295.
INT32	Signed 32-bit integer from two consecutive registers. Range: -2,147,483,648 to 2,147,483,647.
FLOAT32	IEEE 754 single-precision float from two consecutive registers. Shown to 6 significant figures.
FLOAT64	Reserved label for 64-bit doubles requiring four registers. Raw words shown.
HEX	16-bit value in hexadecimal with 16-bit binary representation.

BINARY	16-bit value as a 16-character binary string (MSB first).

NOTE: Changing the Data Type on a connected device triggers a live re-read of the register(s) to refresh the value with the correct interpretation. For 32-bit and float types, two consecutive registers are fetched.

3.6 Encode / Decode Bits Window

Right-click a register row and select Encode / Decode Bits to open the bit manipulation tool. This window allows you to:

- View the 16 individual bits of the register value with their positions labelled (b15 to b0).
- Toggle individual bits by clicking their buttons. The bit grid, all decoded values, and the high/low byte breakdown update instantly.
- View the value simultaneously decoded as UINT16, INT16, HEX, Binary, UINT32 (using the previous register as the high word), INT32, FLOAT32, and bit count.
- Enter any value in decimal, 0xHEX, or binary format in the input field and click Apply to load it into the bit grid.
- Click Clear to reset all bits to zero.

4. Managing the Register Map

4.1 Adding Rows Manually

Click the + Add Row button in the Register Map header to reveal the inline add-row panel. Fill in:

- Name — optional label for the register.
- Modbus Address — the Modbus reference address (e.g. 40001).
- Register Type — select from Coil (FC01), Discrete Input (FC02), Holding Register (FC03), or Input Register (FC04).
- Data Type — initial data type interpretation.

Click Add to insert the row. The value will show 0 (manual) until the next scan refreshes it from the device. Click Cancel or click the button again to dismiss the panel.

4.2 Exporting to CSV

Open the Tools section in the left panel and click Export CSV. A save dialog opens. The exported file contains the following columns:

- MbAddress — Modbus reference address.
- RegType — register type name.
- DataType — current data type.
- Value — last decoded value string.
- Name — label if assigned.
- Changed — last changed indicator.

4.3 Importing from CSV

Click Import CSV in the Tools section to load a previously exported register map. The importer accepts files with MbAddress (or Address), RegType, DataType, Value, and Name columns. RegType can be expressed as the full name (e.g. Holding Register (FC03)) or shorthand (FC03, Holding, HoldingRegister). A confirmation dialog shows the row count before replacing the current map.

WARN: Importing a CSV replaces the entire current register map. Ensure any unsaved work is exported first.

5. Writing to Registers

The Write Register section in the left panel allows individual coil and register writes. Writing requires an active connection.

5.1 Write Single Coil (FC05)

19. Open the Write Register section.
20. Ensure FC05 - Write Coil is selected in the Function dropdown.
21. Enter the zero-based Address of the coil to write.
22. Enter the Value: 1, true, or on to turn the coil ON; 0, false, or off to turn it OFF.
23. Click Write. The debug log confirms the write echo response.

5.2 Write Single Register (FC06)

24. Select FC06 - Write Register in the Function dropdown.
25. Enter the zero-based Address of the holding register.
26. Enter the Value in decimal (e.g. 1500) or hexadecimal (e.g. 0x05DC).
27. Click Write. The debug log shows the echo response confirming the written address and value.

WARN: Writing to registers on a live system can affect device operation. Always confirm the correct address and value before writing. There is no undo.

6. Frame Injector

The Frame Injector is an advanced diagnostic tool for sending custom Modbus frames directly to a connected device. It supports both Structured Builder mode and Raw Hex mode, and works with both Modbus TCP and RTU Tunnel connections.

To open the injector:

28. Expand the Frame Injector section in the left panel (click the purple header).
29. Click the Open Frame Injector button.
30. The injector window displays the current connection mode and Unit ID, and shows CONNECTED or NOT CONNECTED in the top-right badge.

6.1 Structured Builder Mode

Structured mode guides you through frame construction using form fields. The frame preview updates automatically as you type.

Selecting a Function Code

Click one of the six FC tile buttons to select the operation:

Feature / Field	Description
FC01 — Read Coils	Reads one or more coil bits. Enter Start Address and Quantity.
FC02 — Read Discrete Inputs	Reads one or more discrete input bits. Enter Start Address and Quantity.
FC03 — Read Holding Registers	Reads one or more 16-bit holding registers. Enter Start Address and Quantity.
FC04 — Read Input Registers	Reads one or more 16-bit input registers. Enter Start Address and Quantity.
FC05 — Write Single Coil	Writes ON or OFF to a single coil. Enter Address and Value (1=ON, 0=OFF).
FC06 — Write Single Register	Writes a 16-bit value to a single holding register. Enter Address and Value (decimal or 0xHEX).

Frame Preview

The Frame Preview field shows the exact bytes that will be transmitted. For Modbus TCP connections the MBAP header (Transaction ID, Protocol ID, Length, Unit ID) is shown followed by the PDU. For RTU Tunnel connections the raw RTU frame including CRC bytes is shown.

Sending and Reading the Response

Click SEND FRAME to transmit the frame. The Response panel at the bottom of the injector window displays:

- The function code and mode used.

- Round-trip time in milliseconds.
- For read operations: all returned values decoded by register with their Modbus reference address, raw hex, decimal, INT16 interpretation, and individual Hi/Lo bytes.
- For write operations: the echo confirmation and whether the address matches.
- Any Modbus exception codes returned by the device, with a plain-language description.

6.2 Raw Hex Inject Mode

Click the Raw Hex Inject tab to switch to manual byte entry. This mode is for advanced users who need to send non-standard frames or test device edge-case behaviour.

- Enter space-separated hex bytes in the input field (e.g. 01 03 00 00 00 0A).
- The byte counter on the right updates as you type.
- If the Auto-append CRC-16 (RTU) checkbox is ticked and the connection is in RTU Tunnel mode, the tool calculates and appends the correct Modbus CRC-16 before transmitting. The appended CRC bytes are shown in the response log.
- For Modbus TCP connections, include the full MBAP header in your hex bytes. CRC is not used in MBAP mode.
- Click SEND RAW BYTES. The response panel shows the received bytes in hex plus a basic FC and byte-count decode.

NOTE: All injector activity is also mirrored to the main application Debug Log for a unified audit trail.

7. ECY API Integration

The POST Device to API section allows you to register a Modbus device and upload its point list to an ECY REST API endpoint directly from the tool.

7.1 Configuration Fields

Feature / Field	Description
API Base URL	Full URL to the ECY Modbus devices endpoint, e.g. https://192.168.1.1/api/rest/v2/services/modbus/devices
Username	ECY API username (typically admin).
Password	ECY API password. The tool hashes this with SHA-256 before encoding as Basic Auth. Switch to Plain Basic Auth mode in the preview window if your API expects the raw password.

7.2 POST Device

Clicking Preview JSON + POST Device opens a preview window showing the JSON payload that will be sent. The payload includes the device key (Device Name from the Connection section), IP address, TCP port, Unit ID, and default Modbus capability parameters (max read coils, max registers, write support flags).

The preview window allows you to:

- Edit the endpoint URL directly.
- Switch between SHA-256 Basic Auth (default), Plain Basic Auth, or a fully custom authorization header.
- Edit the JSON body before sending.
- Copy the JSON to the clipboard.
- Send the POST request and view the HTTP status code and response body.

7.3 POST Points

Clicking Preview JSON + POST Points generates a JSON object containing all registers currently in the Register Map grid. Each register becomes a named point entry with:

- Key derived from the Name/Label column, or auto-generated as point_<MbAddress> if no label is assigned.
- Modbus address, register type, and data type mapped to ECY API field names.
- Present value, default value, and priority array populated from the last scanned values.
- Access type set to ReadWrite for writable register types and Read for read-only types.

The preview and send workflow is identical to POST Device — review, optionally edit, then transmit.

TIP: Label your registers in the Name/Label column before posting points. The Name value becomes the point key in the ECY system. Meaningful names make the resulting point list far easier to work with.

8. Debug Log

The Debug Log provides a complete protocol trace of all communication between the tool and the device. Click the DEBUG LOG bar at the bottom of the register map to expand or collapse the log panel.

8.1 Log Levels

Feature / Field	Description
[INF]	General application events — connect, disconnect, scan start, import/export.
[OK]	Successful operations — connection established, scan completed, write confirmed.
[ERR]	Errors — connection failures, Modbus exceptions, timeout, CRC mismatch.
[WARN]	Warnings — quantity capped, response UID mismatch, CRC discrepancy.
[HEX]	Raw byte arrays for every transmitted and received frame. Shown in uppercase hex pairs.
[MBAP]	Decoded Modbus frame fields — Transaction ID, Unit ID, FC, address, quantity, byte count, register values.
[TCP]	TCP transport events — connection RTT, bytes sent, bytes received, timing.
[DIAG]	Network diagnostic output — ping result, port open/closed test, NIC details.

NOTE: Debug Mode must be enabled (checkbox in the title bar) for HEX, MBAP, TCP, and DIAG entries to appear. When disabled only INF, OK, ERR, and WARN entries are logged, keeping the output concise during normal operation.

8.2 Saving the Log

Open the Tools section and click Save Log to write the full log contents to a .log text file. Log files can be attached to support requests or used for offline analysis. Click Clear Log to clear the log panel without affecting the register map.

8.3 Network Diagnostic

Click Network Diagnostic in the Connection section to run a quick connectivity test against the configured host and port. The diagnostic performs:

- ICMP ping to check basic IP reachability and measure round-trip time.

- TCP port probe to confirm the Modbus port is open and accepting connections.
- Enumeration of active local network interfaces and their IP addresses to help identify routing issues.

9. Troubleshooting

Feature / Field	Description
Connection times out	Verify the IP address and port. Run Network Diagnostic. Check firewall rules on both the PC and the device. Ensure the device is powered and on the same network segment.
Modbus Exception Code 1 — Illegal Function	The device does not support the selected FC. Try a different register type. Not all devices implement all four FC types.
Modbus Exception Code 2 — Illegal Data Address	The start address or quantity extends beyond the device register map. Reduce the quantity or adjust the start address. Consult the device register map documentation.
Modbus Exception Code 11 — Gateway Target Failed to Respond	The gateway is reachable but the device at the specified Unit ID is not responding. Verify the Unit ID and check the serial bus connection between the gateway and the device.
CRC Mismatch (RTU mode)	Indicates a framing error on the serial side. Check that the gateway baud rate, parity, data bits, and stop bits match the device settings exactly.
No registers returned (scan returns 0 rows)	All returned values may be zero and are hidden. Reduce the start address to 0 and increase the quantity. Try individual FC types rather than ALL.
COM ports list is empty	Your PC does not have a physical serial port or USB-to-serial adapter installed. Use the TCP Tunnel (IP) sub-mode to connect via a serial-to-Ethernet gateway instead.
POST request returns HTTP 401	Authentication failed. In the preview window, switch the Auth Mode to Plain Basic Auth and retry. Some ECY versions do not use SHA-256 password hashing.
POST request returns HTTP 400	The JSON payload is malformed or contains invalid field values. Open the preview window, review the JSON body, and correct any field values before re-sending.

10. Quick Reference

10.1 Workflow Summary

31. Launch the application.
32. Choose connection mode (Modbus TCP or RTU Tunnel) and enter connection details.
33. Click Connect.
34. Set Start Address and Known Register Counts.
35. Select register type (ALL recommended for first discovery) and click Scan.
36. Review the Register Map grid. Label registers using the Name column. Change Data Type for any register that needs a different interpretation.
37. Set a Poll Interval if live monitoring is required; click Scan again to start polling.
38. Use Write Register or Frame Injector as needed.
39. Export CSV for documentation.
40. Fill in API details and use POST Device then POST Points to publish to ECY.

10.2 Tips for Efficient Use

- Run an ALL scan first to discover every active register type before narrowing down to a specific FC.
- Label registers as you discover them — labels persist across re-scans and are preserved in CSV exports.
- Use the Bit Decode window on any status word register to decode individual bit flags without manual calculation.
- Keep Debug Mode enabled when commissioning a new device so you have a complete protocol record. Disable it during routine monitoring to keep the log clean.
- Save the log after any troubleshooting session before clearing it.
- When posting to ECY, use descriptive Name labels (e.g. Supply_Fan_Status rather than point_40001). These become the permanent point keys in the ECY system.
- For RTU Tunnel mode, use the Frame Injector Raw Hex mode with Auto-append CRC enabled to test exact byte sequences when diagnosing gateway issues.

10.3 Modbus Address Reference

FC	Register Type	Access	Address Range	Notes
FC01	Coils	R/W	00001 – 09999	Zero-based addr = ref - 1
FC02	Discrete Inputs	R	10001 – 19999	Zero-based addr = ref - 10001
FC03	Holding Registers	R/W	40001 – 49999	Zero-based addr = ref - 40001
FC04	Input Registers	R	30001 – 39999	Zero-based addr = ref - 30001

11. Glossary

Feature / Field	Description
ADU	Application Data Unit — the complete Modbus frame including header and PDU.
MBAP	Modbus Application Protocol header — the 6-byte TCP wrapper containing Transaction ID, Protocol ID, length, and Unit ID.
PDU	Protocol Data Unit — the core Modbus payload: Function Code plus data bytes.
RTU	Remote Terminal Unit — the serial Modbus framing format using binary encoding and CRC-16 error checking.
CRC-16	Cyclic Redundancy Check — a 2-byte error-detection code appended to every RTU frame. Polynomial: 0xA001, LSB first.
Unit ID	The Modbus device address (1–247). Equivalent to Slave ID in RS-485 networks. Set to 1 for most single-device TCP connections.
FC	Function Code — the one-byte code that specifies the requested Modbus operation (read coils, write register, etc.).
Zero-based address	The internal Modbus register index used in frames. Equal to the Modbus reference number minus the type offset (e.g. ref 40001 = zero-based address 0 for FC03).
ECY	The building automation API target. POST Device and POST Points upload the scanned register map to the ECY system for use as data points.
TCP Tunnel	A serial-to-Ethernet gateway operating in transparent passthrough mode, forwarding raw RTU bytes over TCP without protocol translation.

Modbus Scanner + ECY Point Populator | Version 2.0.0 | User Guide